



西南交通大学  
Southwest Jiaotong University

# 地球科学与环境工程学院

## 《卫星导航定位原理》

### 课间实验报告书

设计题目: 单点定位计算程序编写

学 号: 2023113911

班 级: 遥感 1 班

姓 名: 于沛豪

日 期: 2025. 6. 5

# 目 录

|                           |    |
|---------------------------|----|
| 一、需求分析 .....              | 1  |
| 1.1 设计内容 .....            | 1  |
| 1.2 作业需求 .....            | 1  |
| 1.3 数据文件 .....            | 1  |
| 二、实验原理 .....              | 2  |
| 2.1 坐标计算 .....            | 2  |
| 2.2 接收机钟差改正 .....         | 4  |
| 2.3 电离层误差改正 .....         | 4  |
| 三、解算结果 .....              | 5  |
| 3.1 接收机钟差改正后求得坐标 .....    | 5  |
| 3.2 经对流层、电离层误差改正后坐标 ..... | 7  |
| 四、经验总结 .....              | 8  |
| 五、附录：计算程序 .....           | 9  |
| 5.1 接收机钟差改正后坐标计算程序 .....  | 9  |
| 5.2 经对流层、电离层误差改正后坐标 ..... | 10 |

# 一、需求分析

## 1.1 设计内容

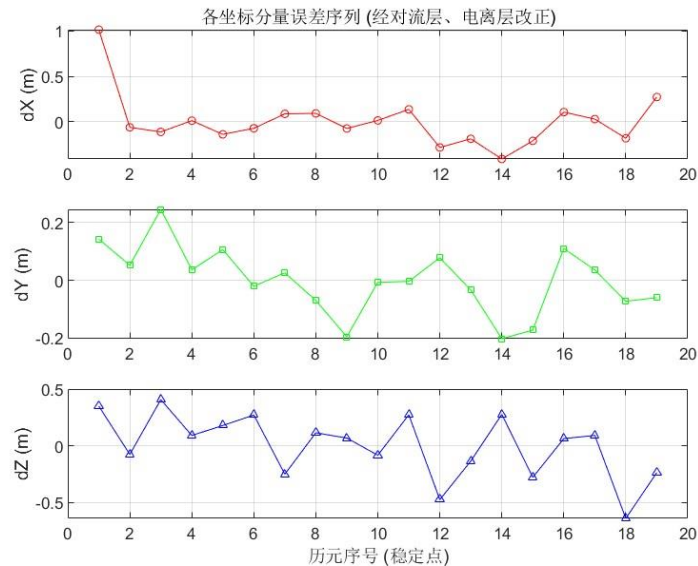


图 1 经接收机钟差、对流层、电离层误差改正后坐标分量误差序列

## 1.2 作业需求

- [1] 根据卫星坐标和伪距观测值，计算 12:00 起测站坐标；
- [2] 完成程序编写并提供计算结果，编程语言不限；
- [3] 撰写定位程序设计报告：包括封面、计算原理（观测方程和解算公式等）、程序 代码、解算结果、经验总结等。

## 1.3 数据文件

- [1] 数据文件 `expt2024.data` 提供的数据包括：地心地固（ECEF）坐标系下的卫星 坐标（第 2,3,4 列）、卫星钟差等误差改正后的测码伪距观测值（第 5 列）和中误差（第 6 列）。测码伪距观测值中接收机钟差、电离层、对流层未改正；
- [2] 测站初始坐标可设为  $(0,0,0)$ ，经过几次循环可收敛到待求坐标点；
- [3] 计算至少五个历元的数据，并将结果列于三线表中。

## 1.4 环境需求

Windows11 以及 MATLAB R2024b。

## 二、实验原理

### 2.1 坐标计算

GNSS 单点定位的基本原理是后方交会。接收机通过测量到至少四颗已知位置卫星的距离，来解算自身的三维坐标和接收机钟差这四个未知数。

伪距（Pseudorange）是最基本的观测量。其简化的数学模型（观测方程）可以表示为

$$P_i = \rho_i + c \cdot (dt_r - dt_i^s) + I_i + T_i + \epsilon_i$$

其中  $P_i$  是接收机对第  $i$  颗卫星的伪距观测量， $\rho_i$  是接收机到第  $i$  颗卫星的真实几何距离。

待求坐标的函数：

$$\rho_i = \sqrt{(X_i^s - X_r)^2 + (Y_i^s - Y_r)^2 + (Z_i^s - Z_r)^2}$$

$(X_i^s, Y_i^s, Z_i^s)$  是第  $i$  颗卫星在 ECEF 坐标系下的已知坐标， $(X_r, Y_r, Z_r)$  是接收机在 ECEF 坐标系下的未知坐标。

其中， $c$  是光速， $dt_r$  是未知的接收机钟差， $dt_i^s$  是已知的卫星钟差， $I_i$  是电高层延迟误差， $T_i$  是对流层延时误差， $\epsilon_i$  是包含了多路径效应和测量噪声的残余误差。

由于式的非线性，我们通常采用泰勒级数展开并线性化的方法，通过迭代最小二乘法进行求解。

首先，对伪距观测方程进行移项，将已知和模型改正项放在左边，未知项和几何距离放在右边：

$$P_i - c \cdot (-dt_i^s) - I_i - T_i = \rho_i + c \cdot dt_r + \epsilon_i$$

$P_i'$  为经过各项误差模型改正后的伪距：

$$P_i' = P_i + c \cdot dt_i^s - I_i - T_i$$

注意：在本次实验中，根据要求说明，输入伪距  $P_i$  已包含了卫星钟差改正  $c \cdot dt_i^s$ ，因此在程序实现中，我们仅对电离层和对流层进行改正。

将式在接收机近似坐标  $(X_{r0}, Y_{r0}, Z_{r0})$  处进行一阶泰勒展开，并整理后可得到线性化的观测方程。写成矩阵形式为：

$$\mathbf{v} = \mathbf{B}\mathbf{x} - \mathbf{l}$$

其中各个向量和矩阵的定义如下：

$\mathbf{x}$  是待求参数的修正量向量：

$$\mathbf{x} = \begin{bmatrix} \Delta X_r \\ \Delta Y_r \\ \Delta Z_r \\ c \cdot dt_r \end{bmatrix}$$

$\mathbf{l}$  是观测值减去近似几何距离的残差向量，其每个元素为：

$$l_i = P_i' - \rho_{i0}$$

这里的  $\rho_{i0}$  是用近似坐标  $(X_{r0}, Y_{r0}, Z_{r0})$  计算出的几何距离。

$\mathbf{B}$  是设计矩阵：

$$\mathbf{B} = \begin{pmatrix} -l_{ix} & -l_{iy} & -l_{iz} & 1 \\ \vdots & \vdots & \vdots & \vdots \\ -l_{nx} & -l_{ny} & -l_{nz} & 1 \end{pmatrix}$$

矩阵中的  $l_{ix}, l_{iy}, l_{iz}$  分别是接收机到第  $i$  颗卫星的单位视线向量的三个分量：

$$l_{ix} = \frac{X_i^s - X_{r0}}{\rho_{i0}}, l_{iy} = \frac{Y_i^s - Y_{r0}}{\rho_{i0}}, l_{iz} = \frac{Z_i^s - Z_{r0}}{\rho_{i0}}$$

根据最小二乘原理 ( $\mathbf{v}^T \mathbf{v} = \min$ )，可得参数估值的法方程解：

$$\hat{\mathbf{x}} = (\mathbf{B}^T \mathbf{B})^{-1} \mathbf{B}^T \mathbf{l}$$

得到修正量后，更新接收机的近似坐标：

$$\begin{pmatrix} X_r \\ Y_r \\ Z_r \\ c \cdot dt_r \end{pmatrix}_{k+1} = \begin{pmatrix} X_r \\ Y_r \\ Z_r \\ c \cdot dt_r \end{pmatrix}_k + \hat{\mathbf{x}}$$

重复上述过程进行迭代，直到修正量  $\hat{\mathbf{x}}$  的范数小于预设的阈值，迭代收敛。

## 2.2 接收机钟差改正

接收机钟差  $dt_r$  是由于接收机内部的晶振与精确的 GPS 时间系统 (GPST) 不同步而产生的。它对所有卫星的使距观测产生一个公共的偏差。如式所示, 它在设计矩阵  $B$  中对应的系数恒为 1。因此, 我们无法通过模型预先改正它, 而是必须将其作为第 4 个未知数, 与三维坐标一起在最小二乘平差中进行求解。

## 2.3 电离层误差改正

电离层延迟是由于 GPS 信号在穿过地球上空 60—1000 km 的电离层时, 受到自由电子影响而产生的路径延迟。本实验采用 GPS 广播星历中提供的 Klobuchar 模型进行改正。该模型是一个经验模型, 能够改正约 50—60% 的电离层延迟。

其核心思想是, 用一个余弦函数来模拟白天电离层延迟的变化, 而在夜间则认为是一个常数。最终的电离层延迟距离  $I$  (单位: 米) 的计算公式可以简化表示为:

$$I = c \cdot F \cdot \left( A_1 + A_2 \cdot \cos \left( \frac{2\pi(t - t_p)}{P} \right) \right)$$

其中  $F$  是斜向因子, 用于将天顶方向的延迟转换为卫星视线方向的延迟, 它是卫星高度角的函数。 $A_1$  代表夜间延迟的常数值 (5 纳秒)。

$A_2$  是余弦函数的幅度,  $P$  是周期。 $A_2$  和  $P$  都是根据广播星历中的 8 个参数 ( $\alpha_0, \dots, \alpha_3, \beta_0, \dots, \beta_3$ ) 以及接收机和卫星的几何关系计算得出的。 $t$  是地方时,  $t_p$  是延迟峰值出现的地方时 (通常是 14:00)。

## 2.4 对流层误差改正

对流层延迟是由于 GPS 信号在穿过地球表面以上至约 40 km 的对流层时, 受到大气折射而产生的路径延迟。本实验采用经典的 Saastamoinen 模型进行改正。

该模型将对流层延迟分为干延迟和湿延迟两部分, 并通过一个映射函数将其从天顶方向转换到卫星视线方向。其表达式为:

$$T = T_{z,dry} \cdot M_{dry}(El) + T_{z,wet} \cdot M_{wet}(El)$$

通常可以简化为:

$$T = (T_{z,dry} + T_{z,wet}) \cdot M(El)$$

其中 $T_{z,dry}$  和  $T_{z,wet}$  分别是天顶方向的干、湿延迟分量。它们是测站处的大气压力  $P$  、温度  $T$  和水汽压  $e$  的函数。在没有实测气象数据时，通常采用标准大气模型根据观站高程  $H$  和纬度  $\phi$  来估算。 $M(El)$  是映射函数，通常可近似为  $1/\sin(El)$ ，其中  $El$  是卫星高度角。

本程序中使用的 Saastamoinen 模型即基于上述原理，利用标准大气参数估算对流层天顶总延迟，再投影到卫星方向上。

### 三、解算结果

#### 3.1 接收机钟差改正后求得坐标

选取 12:00 到 12:30 的观测数据，选取前 20 个数据做展示，如下所示

表 1 接收机钟差误差改正后求得坐标

| 分钟 | X                | Y                | Z                |
|----|------------------|------------------|------------------|
| 0  | 4331314.20107695 | 567539.988513713 | 4633148.78662010 |
| 1  | 4331313.86868085 | 567539.695828074 | 4633148.07445924 |
| 2  | 4331313.27317716 | 567539.629747338 | 4633147.63395068 |
| 3  | 4331313.18737093 | 567539.770144318 | 4633147.99988483 |
| 4  | 4331313.28134610 | 567539.508879879 | 4633147.56993241 |
| 5  | 4331313.10402422 | 567539.528458672 | 4633147.54822987 |
| 6  | 4331313.14895608 | 567539.352467147 | 4633147.53526007 |
| 7  | 4331313.29142298 | 567539.351164648 | 4633146.91066331 |
| 8  | 4331313.28361203 | 567539.209028268 | 4633147.17802738 |
| 9  | 4331313.10855418 | 567539.037830172 | 4633147.03767935 |
| 10 | 4331314.20107695 | 567539.988513713 | 4633148.78662010 |
| 11 | 4331313.86868085 | 567539.695828074 | 4633148.07445924 |
| 12 | 4331313.27317716 | 567539.629747338 | 4633147.63395068 |
| 13 | 4331313.18737093 | 567539.770144318 | 4633147.99988483 |
| 14 | 4331313.28134610 | 567539.508879879 | 4633147.56993241 |
| 15 | 4331313.10402422 | 567539.528458672 | 4633147.54822987 |
| 16 | 4331313.14895608 | 567539.352467147 | 4633147.53526007 |
| 17 | 4331313.29142298 | 567539.351164648 | 4633146.91066331 |
| 18 | 4331313.28361203 | 567539.209028268 | 4633147.17802738 |
| 19 | 4331313.10855418 | 567539.037830172 | 4633147.03767935 |
| 20 | 4331314.20107695 | 567539.988513713 | 4633148.78662010 |

接收机钟差变化趋势如下所示：

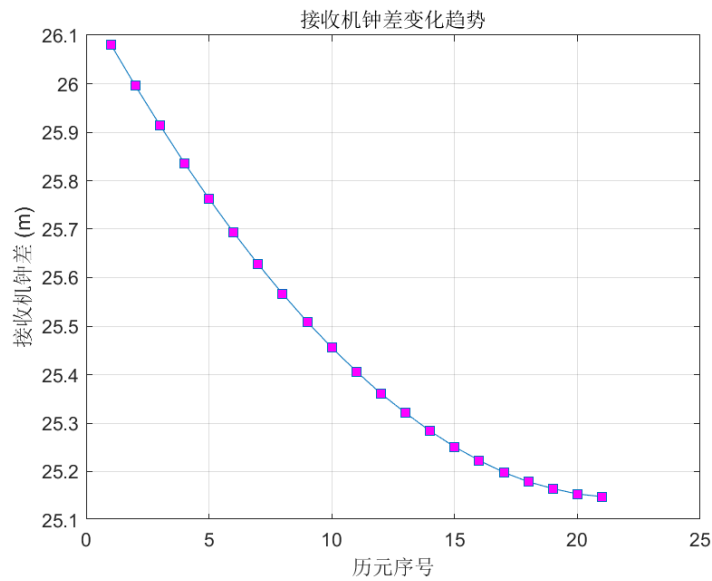


图2 接收机钟差变化趋势

由此，我们可以计算出坐标分量（相对于平均值），如下图：

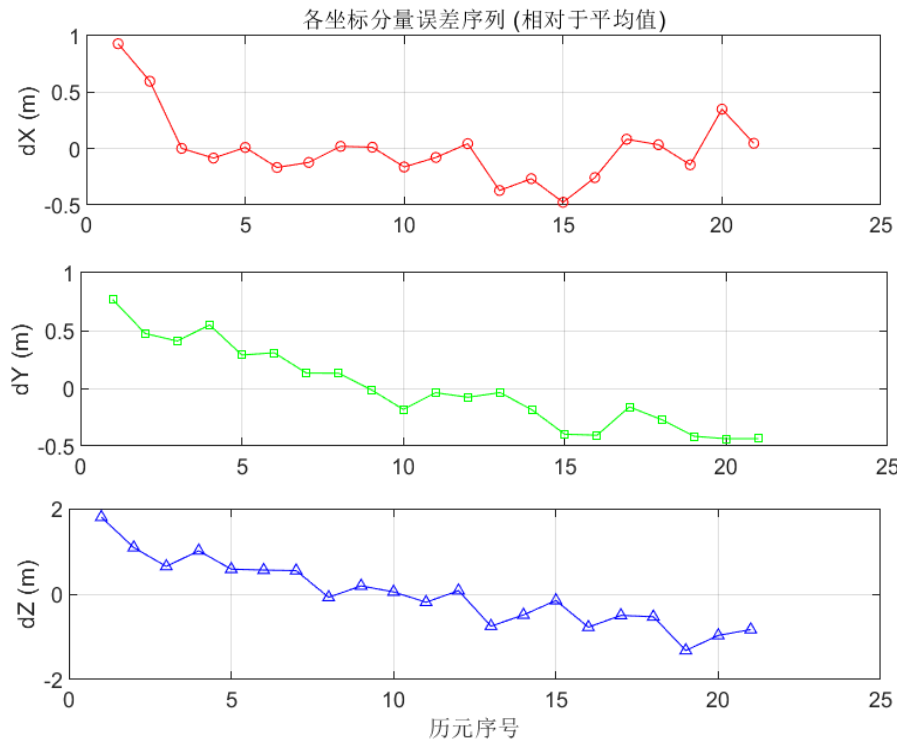


图3 各坐标分量的误差序列

具体检核数据如下：

- 时段内平均接收机位置：4331313.271, 567539.221, 4633146.986
- 时段内平均接收机钟差：25.483 m (85.001 ns)
- 定位结果标准差 (X, Y, Z)：0.313, 0.357, 0.782 m



### 3.2 经对流层、电离层误差改正后坐标

在接收机钟差改正后，我们可以利用上述模型编写代码，对对流层、电离层误差改正后，得出坐标，如下所示

表 1 经接收机钟差、对流层、电离层误差改正后坐标

| 分钟 | X                | Y                | Z                |
|----|------------------|------------------|------------------|
| 0  | 4331302.07585634 | 567537.764341440 | 4633135.08745784 |
| 1  | 4331300.99741920 | 567537.673529060 | 4633134.66090872 |
| 2  | 4331300.94690600 | 567537.867369550 | 4633135.14475236 |
| 3  | 4331301.07124167 | 567537.657941425 | 4633134.82833789 |
| 4  | 4331300.91999202 | 567537.727947722 | 4633134.91635391 |
| 5  | 4331300.98673005 | 567537.601030057 | 4633135.00929070 |
| 6  | 4331301.14628040 | 567537.647395775 | 4633134.48638087 |
| 7  | 4331301.15167588 | 567537.551745050 | 4633134.85192515 |
| 8  | 4331300.98505462 | 567537.425708292 | 4633134.80547835 |
| 9  | 4331301.07368678 | 567537.614425913 | 4633134.65371333 |
| 10 | 4331301.19594137 | 567537.617897764 | 4633135.01118610 |
| 11 | 4331300.77504993 | 567537.699718861 | 4633134.26142976 |
| 12 | 4331300.87089247 | 567537.587861239 | 4633134.60263327 |
| 13 | 4331300.64776938 | 567537.419187521 | 4633135.01413285 |
| 14 | 4331300.84803856 | 567537.449060652 | 4633134.45887147 |
| 15 | 4331301.16543406 | 567537.731859854 | 4633134.80066152 |
| 16 | 4331301.08847568 | 567537.657344646 | 4633134.82889869 |
| 17 | 4331300.87672983 | 567537.548932085 | 4633134.09618246 |
| 18 | 4331301.33229131 | 567537.561481193 | 4633134.50080650 |
| 19 | 4331302.07585634 | 567537.764341440 | 4633135.08745784 |
| 20 | 4331300.99741920 | 567537.673529060 | 4633134.66090872 |

具体检核数据如下：

- 时段内平均接收机位置：4331301.061, 567537.621, 4633134.738
- 时段内平均接收机钟差：4.011 m (13.378 ns)
- 定位结果标准差 (X, Y, Z)：0.295, 0.116, 0.284 m

同上，计算出坐标分量的平均值，如下图：

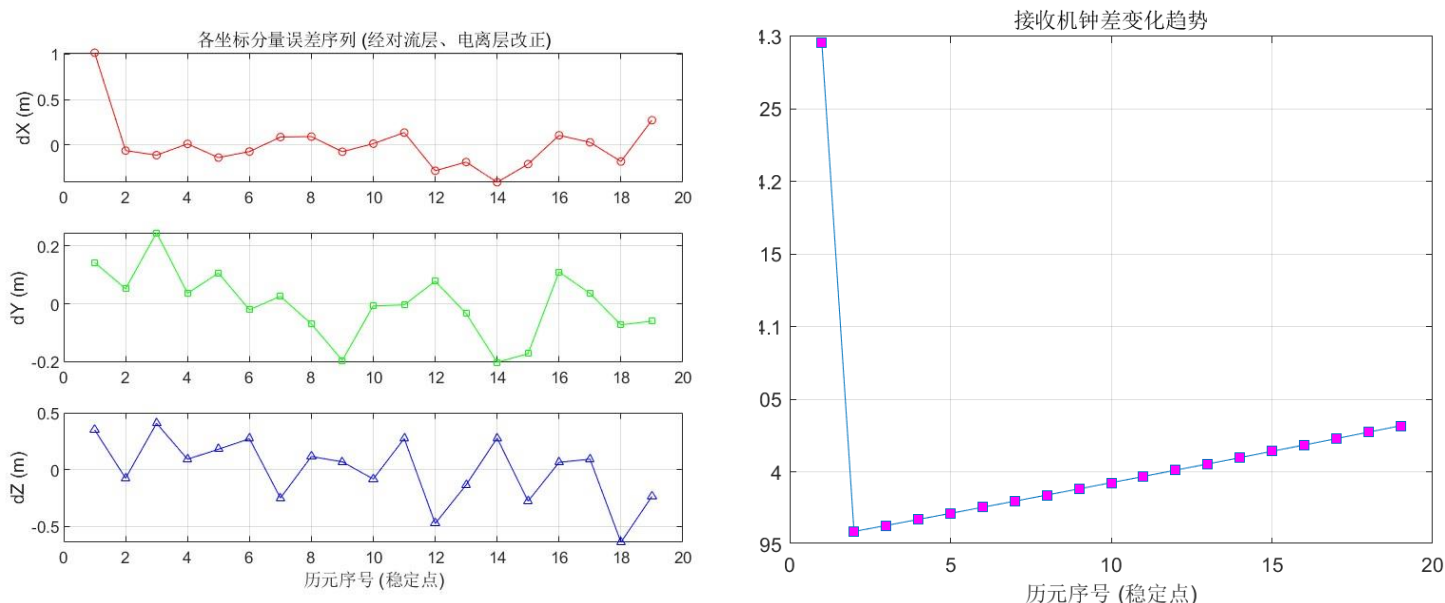


图 4 经接收机钟差、对流层、电离层误差改正后坐标分量误差序列

#### 四、经验总结

本次实验通过使用 MATLAB 编程,成功实现了基于伪距观测值的单点定位(SPP)算法。通过对实验的全过程,我有一些深刻的体会与总结:

实验让我从代码层面真正掌握了伪距单点定位的数学模型。我清晰地理解了如何构建观测方程,如何将非线性的距离方程通过泰勒级数展开进行线性化,以及如何构造设计矩阵(B 矩阵)和观测值减计算值向量(L 向量),最终通过迭代最小二乘法求解接收机三维坐标和钟差这四个未知数。

根据实验要求,输入数据未改正电离层和对流层延迟。在初步实现基本定位算法后,我意识到若不考虑这两项误差,其量级可达数米,将严重影响定位精度。这促使我进一步学习并集成了 Klobuchar 电离层模型和 Saastamoinen 对流层模型,使定位结果更接近真实情况,也让我直观地感受到了 GNSS 误差模型的理论价值与实际意义。

我熟练掌握了在 MATLAB 中进行数据读取、矩阵运算、循环迭代以及函数化编程的技巧。特别是 calculate\_spp 函数的编写,它完整地封装了从数据准备、迭代解算到收敛判断的整个流程,体现了模块化设计的思想。

通过对多个历元的解算结果进行处理,我学会了计算定位结果的均值和标准差,用以评估定位的精度和稳定性。利用 MATLAB 强大的绘图功能,我将各坐标分量相对于均值的误差序列以及接收机钟差的变化趋势进行了可视化,这种方式比单纯的数字表格更能直观地揭示定位的特征。

从查文档了解各个参数的意义,到完成程序编写后,直到最后输出图像的时候,那一刻的自豪感真是无可比拟。程序已随实验报告打包提交。

## 五、附录：计算程序

### 5.1 接收机钟差改正后坐标计算程序

通过读取一个包含多个历元卫星观测数据（主要是卫星三维坐标和伪距）的文本文件，对每一个历元的数据进行处理。采用迭代最小二乘法，当观测到至少四颗卫星时，就能解算出接收机的三维位置（X，Y，Z）以及接收机的钟差。

```
function gnss_spp_core()
    c = 299792458;
    pos_guess = [0; 0; 0; 0];
    pos_solution = [];

    file_id = fopen('data.txt', 'r');
    if file_id == -1, error('无法打开文件 data.txt。'); end

    current_epoch_info = struct('prn', {}, 'data', {});

    while ~feof(file_id)
        line = fgetl(file_id);
        if ~ischar(line) || isempty(strtrim(line)), continue; end

        if startsWith(strtrim(line), '>')
            if ~isempty(current_epoch_info)
                [pos_solution, success] = calculate_spp(current_epoch_info, pos_guess, c);
                if success
                    pos_guess = pos_solution;
                end
            end
            current_epoch_info = struct('prn', {}, 'data', {});
        else
            prn = sscanf(line, '%s', 1);
            data = sscanf(line(4:end), '%f');
            if numel(data) == 5
                current_epoch_info(end+1).prn = prn;
                current_epoch_info(end).data = data;
            end
        end
    end

    if ~isempty(current_epoch_info)
        [pos_solution, success] = calculate_spp(current_epoch_info, pos_guess, c);
    end

    fclose(file_id);

    if ~isempty(pos_solution)
        fprintf('\n=====');
        fprintf('最终解算结果:\n');
```

```

    fprintf('位置 (X, Y, Z): %.3f, %.3f, %.3f m\n', pos_solution(1:3));
    fprintf('接收机钟差: %.3f m (%.3f ns)\n', pos_solution(4), pos_solution(4)/c*1e9);
    fprintf('=====\n');
else
    disp('所有历元均定位失败。');
end
end

function [final_pos, success] = calculate_spp(epoch_info, initial_pos, c)
    MAX_ITER = 15;
    CONV_THRESH = 1e-5;
    num_sats = numel(epoch_info);
    final_pos = initial_pos;
    success = false;

    if num_sats < 4, return; end

    sat_xyz = zeros(num_sats, 3);
    pseudo_ranges = zeros(num_sats, 1);
    for i = 1:num_sats
        sat_xyz(i, :) = epoch_info(i).data(1:3);
        pseudo_ranges(i) = epoch_info(i).data(4);
    end

    pos = initial_pos;
    for iter = 1:MAX_ITER
        p0 = sqrt(sum((sat_xyz - pos(1:3)).^2, 2));
        los_vectors = (sat_xyz - pos(1:3)) ./ p0;

        B = [-los_vectors, ones(num_sats, 1)];
        L = pseudo_ranges - p0 - pos(4);

        deltap = (B' * B) \ (B' * L);
        pos = pos + deltap;

        if norm(deltap(1:3)) < CONV_THRESH
            final_pos = pos;
            success = true;
            return;
        end
    end

    final_pos = pos;
    success = false;
end

```

## 5.2 经对流层、电离层误差改正后坐标

与基础版本相比，该脚本引入了对主要误差源的改正模型，显著提升了定位的准确性。具体来说，脚本在计算伪距时，会主动估算并扣除由地球大气层引起的信号传播延迟。

```

function gnss_spp_atmospheric_core()
    c = 299792458;
    pos_guess = [0; 0; 0; 0];
    pos_solution = [];

    iono_params.alpha = [0.1118E-07, 0.1490E-07, -0.5960E-07, -0.1192E-06];
    iono_params.beta = [0.1167E+06, 0.1311E+06, -0.1966E+06, -0.3277E+06];

    file_id = fopen('data.txt', 'r');
    if file_id == -1, error('无法打开文件 data.txt。'); end

    current_epoch_info = struct('prn', {}, 'data', {});
    current_epoch_time = [];

    while ~feof(file_id)
        line = fgetl(file_id);
        if ~ischar(line) || isempty(strtrim(line)), continue; end

        if startsWith(strtrim(line), '>')
            if ~isempty(current_epoch_info)
                [pos_solution_new, success] = calculate_spp_core(current_epoch_info, pos_guess, iono_params, current_epoch_time, c);
                if success
                    pos_guess = pos_solution_new;
                    pos_solution = pos_solution_new; % Update final solution
                end
            end
            time_data = sscanf(line, '>%d %d %d %d %d %f');
            current_epoch_time = datetime(time_data(1:6));
            current_epoch_info = struct('prn', {}, 'data', {});
        else
            prn = sscanf(line, '%s', 1);
            data = sscanf(line(4:end), '%f');
            if numel(data) == 5
                current_epoch_info(end+1).prn = prn;
                current_epoch_info(end).data = data;
            end
        end
    end

    if ~isempty(current_epoch_info)
        [pos_solution, ~] = calculate_spp_core(current_epoch_info, pos_guess, iono_params, current_epoch_time, c);
    end
    fclose(file_id);

    if ~isempty(pos_solution)
        fprintf('\n--- 最终解算结果 (含大气改正) ---\n');
        fprintf('位置 (X, Y, Z): %.3f, %.3f, %.3f m\n', pos_solution(1:3));
        fprintf('接收机钟差: %.3f m (%.3f ns)\n', pos_solution(4), pos_solution(4)/c*1e9);
    else
        disp('所有历元均定位失败。');
    end
end

function [final_pos, success] = calculate_spp_core(epoch_info, initial_pos, iono_params, t_rx, c)
    MAX_ITER = 15;
    CONV_THRESH = 1e-5;
    ELEVATION_MASK = 15;

```

```

pos = initial_pos;
final_pos = initial_pos;
success = false;

is_first_iter_at_origin = (norm(pos(1:3)) < 1.0);
valid_sat_indices = [];
if is_first_iter_at_origin
    valid_sat_indices = 1:numel(epoch_info);
else
    for i = 1:numel(epoch_info)
        [~, el_deg, ~] = ecef2azel(pos(1:3)', epoch_info(i).data(1:3)');
        if el_deg >= ELEVATION_MASK
            valid_sat_indices(end+1) = i;
        end
    end
end

num_valid_sats = numel(valid_sat_indices);
if num_valid_sats < 4, return; end

for iter = 1:MAX_ITER
    valid_sat_xyz = zeros(num_valid_sats, 3);
    proc_ranges = zeros(num_valid_sats, 1);

    for k = 1:num_valid_sats
        idx = valid_sat_indices(k);
        sat_pos = epoch_info(idx).data(1:3)';
        raw_range = epoch_info(idx).data(4);
        ion_delay = 0;
        trop_delay = 0;

        if ~is_first_iter_at_origin
            [lat_u, lon_u, h_u] = ecef2geodetic_core(pos(1:3)');
            [az, el, ~] = ecef2azel(pos(1:3)', sat_pos);
            t_gps = 3600*hour(t_rx) + 60*minute(t_rx) + second(t_rx);
            ion_delay = klobuchar_model_core(lat_u, lon_u, el, az, t_gps, iono_params,
c);
            trop_delay = saastamoinen_model_core(h_u, el);
        end

        valid_sat_xyz(k,:) = sat_pos;
        proc_ranges(k) = raw_range - ion_delay - trop_delay;
    end

    p0 = sqrt(sum((valid_sat_xyz - pos(1:3)').^2, 2));
    los_vectors = (valid_sat_xyz - pos(1:3)') ./ p0;
    B = [-los_vectors, ones(num_valid_sats, 1)];
    L = proc_ranges - p0 - pos(4);

    deltap = pinv(B) * L;
    pos = pos + deltap;

    if norm(deltap(1:3)) < CONV_THRESH
        final_pos = pos; success = true; return;
    end
end
final_pos = pos;
end

function [lat, lon, h] = ecef2geodetic_core(ecef)

```

```

a = 6378137.0; f = 1/298.257223563; e2 = 2*f - f^2;
x=ecef(1); y=ecef(2); z=ecef(3);
p = sqrt(x^2 + y^2); lon = atan2(y, x);
lat_old = atan2(z, p * (1 - e2)); h_old = 0;
for i=1:5, N = a/sqrt(1-e2*sin(lat_old)^2); h = p/cos(lat_old)-N; lat = atan2(z, p*(1-
e2*N/(N+h))); if abs(lat-lat_old)<1e-12&&abs(h-h_old)<1e-
12, break; end, lat_old=lat; h_old=h; end
lat = rad2deg(lat); lon = rad2deg(lon);
end

function [az, el, dist] = ecef2azel(user_pos, sat_pos)
[lat, lon, ~] = ecef2geodetic_core(user_pos);
lat_rad = deg2rad(lat); lon_rad = deg2rad(lon);
R = [-sin(lon_rad),cos(lon_rad),0; -sin(lat_rad)*cos(lon_rad),-
sin(lat_rad)*sin(lon_rad),cos(lat_rad); cos(lat_rad)*cos(lon_rad),cos(lat_rad)*sin(lon_rad)
,sin(lat_rad)];
d_ecef = (sat_pos - user_pos)';
d_enu = R * d_ecef;
E=d_enu(1); N=d_enu(2); U=d_enu(3);
dist = norm(d_enu); el = rad2deg(atan2(U, sqrt(E^2 + N^2)));
az = rad2deg(atan2(E, N)); if az < 0, az = az + 360; end
end

function ion_delay = klobuchar_model_core(lat_u, lon_u, el, az, t_gps, iono, c)
el_sc = el/180; lat_u_sc=lat_u/180; lon_u_sc=lon_u/180; az_rad=deg2rad(az);
psi = 0.0137/(el_sc+0.11)-0.022;
lat_ipp = lat_u_sc+psi*cos(az_rad);
if lat_ipp>0.416, lat_ipp=0.416; elseif lat_ipp<-0.416, lat_ipp=-0.416; end
lon_ipp = lon_u_sc+(psi*sin(az_rad))/cos(lat_ipp*pi);
lat_m = lat_ipp+0.064*cos((lon_ipp-1.617)*pi);
t = mod(4.32e4*lon_ipp+t_gps, 86400);
AMP = sum(iono.alpha .* [1, lat_m, lat_m^2, lat_m^3]); if AMP<0, AMP=0; end
PER = sum(iono.beta .* [1, lat_m, lat_m^2, lat_m^3]); if PER<72000, PER=72000; end
x = 2*pi*(t-50400)/PER; F = 1.0+16.0*(0.53-el_sc)^3;
if abs(x)<1.57, ion_delay=c*F*(5e-9+AMP*(1-x^2/2+x^4/24)); else, ion_delay=c*F*5e-
9; end
end

function trop_delay = saastamoinen_model_core(h, el)
if h<-1000|h>10000, h=0; end
P = 1013.25 * (1 - 2.26e-5 * h)^5.225;
T = 288.15 - 6.5e-3 * h;
e = 0.5 * exp(-37.2465 + 0.213166*T - 0.000256908*T^2);
zd = 0.002277 * (P + (0.05 + 1255/T) * e);
el_rad = deg2rad(el); if el_rad<=0, el_rad = 1e-6; end
trop_delay = zd/sin(el_rad);
end

```